User Manual: Agent Checkpointing System

Complete Guide to Installation, Configuration, and Usage

Document Version: 1.0.0 **Last Updated:** 2026-01-25 **Minimum Requirements:** Node.js 20+, MongoDB 7.0+

Table of Contents

1. Installation and Setup
2. API Reference
3. Recovery Scenarios
4. Troubleshooting

---

1. Installation and Setup

1.1 Prerequisites

Before installation, ensure you have:

Requirement	Minimum Version	Verification Command
Node.js	20.0.0+	node –version
npm	10.0.0+	npm –version
MongoDB	7.0+	mongod –version
TypeScript	5.3+	npx tsc –version

1.2 Installation Steps

Step 1: Navigate to Project Directory

```
cd candidates/candidate-4-checkpointing
```

Step 2: Install Dependencies

```
npm install
```

This installs: - express (4.x) - HTTP server - mongoose (8.x) - MongoDB ODM - zod (3.x) - Schema validation - uuid (9.x) - UUID generation - dotenv (16.x) - Environment configuration

Step 3: Configure Environment

Create a .env file in the project root:

```
MONGODB_URI=mongodb://localhost:27017/checkpointing
PORT=3002
NODE_ENV=development
```

1.3 MongoDB Setup

Option A: Using Docker (Recommended)

```
# Start MongoDB container
docker run -d \
  --name mongodb \
  -p 27017:27017 \
  -v mongodb_data:/data/db \
  mongo:7.0
```

```
# Verify it is running
docker ps | grep mongodb
```

Option B: Local MongoDB Installation

For macOS with Homebrew:

```
brew tap mongodb/brew
brew install mongodb-community@7.0
brew services start mongodb-community@7.0
```

For Ubuntu/Debian:

```
wget -qO- https://www.mongodb.org/static/pgp/server-7.0.asc | sudo tee /etc/apt/
trusted.gpg.d/mongodb.asc
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-
org/7.0 multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt update
sudo apt install -y mongodb-org
sudo systemctl start mongod
```

Verify MongoDB Connection:

```
mongosh --eval "db.adminCommand('ping')"
# Should output: { ok: 1 }
```

1.4 Starting the Server

Development Mode (with auto-reload):

```
npm run dev
```

Production Mode:

```
npm run build
npm start
```

Verify Server is Running:

```
curl http://localhost:3002/health
# Expected: {"status":"ok","timestamp":"..."}
```

1.5 Directory Structure After Setup

```
candidate-4-checkpointing/
├── node_modules/           # Dependencies
├── src/
│   ├── api/               # REST API routes
│   ├── domain/            # Domain models
│   ├── managers/          # Business logic
│   ├── recovery/          # Recovery logic
│   ├── serialization/     # JSON serialization
│   ├── storage/           # MongoDB persistence
│   ├── config/            # Configuration
│   └── index.ts           # Entry point
├── tests/                 # Unit tests
├── docs/                  # Documentation
├── .env                   # Environment config
├── package.json
└── tsconfig.json
```

---

2. API Reference

2.1 API Overview

Endpoint	Method	Description
/api/checkpoints	POST	Create checkpoint
/api/checkpoints/:agentId	GET	List checkpoints
/api/checkpoints/:agentId/latest	GET	Get latest checkpoint
/api/checkpoints/:checkpointId	DELETE	Delete checkpoint
/api/checkpoints/:agentId/all	DELETE	Delete all agent checkpoints
/api/recovery/recover	POST	Recover agent
/api/recovery/points/:agentId	GET	List recovery points
/api/recovery/validate/:checkpointId	POST	Validate checkpoint

2.2 Create Checkpoint

Endpoint: POST /api/checkpoints

Request Body:

```
{
  "agentId": "123e4567-e89b-12d3-a456-426614174000",
  "state": {
    "messages": [
      {
        "role": "user",
        "content": "Hello",
        "timestamp": "2026-01-25T10:00:00Z"
      },
      {
        "role": "assistant",
        "content": "Hi there!",
        "timestamp": "2026-01-25T10:00:01Z"
      }
    ],
    "variables": {
      "counter": 5,
      "userName": "Alice",
      "results": ["item1", "item2"]
    },
    "executionPosition": {
      "step": 3,
      "functionName": "processData"
    },
    "status": "running"
  },
  "options": {
    "type": "full",
    "description": "Before database operation",
    "tags": ["important", "pre-db"],
    "reason": "manual",
```

```
    "ttl": 3600
  }
}
```

Request Fields:

Field	Type	Required	Description
agentId	UUID	Yes	Unique identifier for the agent
state	Object	Yes	Complete agent state
state.messages	Array	Yes	Conversation history
state.variables	Object	Yes	Agent working memory
state.executionPosition	Object	No	Current execution context
state.status	String	No	Agent status (default: idle)
options	Object	No	Checkpoint creation options
options.type	String	No	full or incremental
options.description	String	No	Human-readable description
options.tags	Array	No	Searchable tags
options.reason	String	No	Why checkpoint was created
options.ttl	Number	No	Seconds until expiration

Success Response (201):

```
{
  "success": true,
  "checkpoint": {
    "checkpointId": "456e7890-e12b-34c5-d678-901234567890",
    "agentId": "123e4567-e89b-12d3-a456-426614174000",
    "timestamp": "2026-01-25T10:00:00.000Z",
    "type": "full",
    "size": 1024,
    "sequenceNumber": 1
  }
}
```

Skipped Response (200):

When no state changes detected:

```
{
  "success": true,
  "skipped": true,
  "reason": "No state changes since last checkpoint"
}
```

Error Response (400):

```
{
  "success": false,
  "error": "Invalid request body",
  "details": [
    {
      "path": ["agentId"],
      "message": "Invalid uuid"
    }
```

```
  ]
}
```

2.3 List Checkpoints

Endpoint: GET /api/checkpoints/:agentId

Query Parameters:

Parameter	Type	Default	Description
limit	Number	10	Maximum checkpoints to return

Example Request:

```
curl "http://localhost:3002/api/checkpoints/123e4567-e89b-12d3-a456-426614174000?
limit=5"
```

Success Response (200):

```
{
  "success": true,
  "checkpoints": [
    {
      "checkpointId": "...",
      "timestamp": "2026-01-25T10:00:00Z",
      "type": "full",
      "size": 1024,
      "sequenceNumber": 3
    },
    {
      "checkpointId": "...",
      "timestamp": "2026-01-25T09:55:00Z",
      "type": "incremental",
      "size": 256,
      "sequenceNumber": 2
    }
  ],
  "stats": {
    "totalCount": 3,
    "fullCount": 2,
    "incrementalCount": 1,
    "totalSize": 2304,
    "latestTimestamp": "2026-01-25T10:00:00Z"
  }
}
```

2.4 Get Latest Checkpoint

Endpoint: GET /api/checkpoints/:agentId/latest

Example Request:

```
curl http://localhost:3002/api/checkpoints/123e4567-e89b-12d3-a456-426614174000/
latest
```

Success Response (200):

```
{
  "success": true,
  "checkpoint": {
    "checkpointId": "456e7890-e12b-34c5-d678-901234567890",
```

```
    "agentId": "123e4567-e89b-12d3-a456-426614174000",
    "timestamp": "2026-01-25T10:00:00Z",
    "state": {
      "messages": [...],
      "variables": {...},
      "status": "running"
    },
    "type": "full",
    "size": 1024,
    "sequenceNumber": 3,
    "metadata": {
      "description": "Before database operation",
      "tags": ["important"],
      "checkpointReason": "manual"
    }
  }
}
```

Not Found Response (404):

```
{
  "success": false,
  "error": "No checkpoints found for agent"
}
```

2.5 Recover Agent

Endpoint: POST /api/recovery/recover

Request Body:

```
{
  "agentId": "123e4567-e89b-12d3-a456-426614174000",
  "checkpointId": "456e7890-e12b-34c5-d678-901234567890",
  "verifyIntegrity": true,
  "fallbackToLatest": true
}
```

Request Fields:

Field	Type	Required	Description
agentId	UUID	Yes	Agent to recover
checkpointId	UUID	No	Specific checkpoint (default: latest)
verifyIntegrity	Boolean	No	Verify before restore (default: true)
fallbackToLatest	Boolean	No	Try next checkpoint on failure (default: true)

Success Response (200):

```
{
  "success": true,
  "agentId": "123e4567-e89b-12d3-a456-426614174000",
  "checkpointId": "456e7890-e12b-34c5-d678-901234567890",
  "restoredState": {
    "messages": [...],
```

```
    "variables": {...},
    "status": "paused"
  },
  "recoveryTime": 15
}
```

Error Response (400):

```
{
  "success": false,
  "error": "No valid checkpoints found"
}
```

2.6 Validate Checkpoint

Endpoint: POST /api/recovery/validate/:checkpointId

Example Request:

```
curl -X POST http://localhost:3002/api/recovery/validate/456e7890-e12b-34c5-d678-901234567890
```

Success Response (200):

```
{
  "success": true,
  "checkpointId": "456e7890-e12b-34c5-d678-901234567890",
  "valid": true,
  "issues": []
}
```

Validation Failure Response (200):

```
{
  "success": true,
  "checkpointId": "456e7890-e12b-34c5-d678-901234567890",
  "valid": false,
  "issues": [
    "Checkpoint has expired",
    "Base checkpoint for incremental checkpoint not found"
  ]
}
```

2.7 Delete Checkpoint

Endpoint: DELETE /api/checkpoints/:checkpointId

Example Request:

```
curl -X DELETE http://localhost:3002/api/checkpoints/456e7890-e12b-34c5-d678-901234567890
```

Success Response (200):

```
{
  "success": true,
  "message": "Checkpoint deleted"
}
```

2.8 Delete All Agent Checkpoints

Endpoint: DELETE /api/checkpoints/:agentId/all

Example Request:

```
curl -X DELETE http://localhost:3002/api/checkpoints/123e4567-e89b-12d3-a456-426614174000/all
```

Success Response (200):

```
{
  "success": true,
  "message": "Deleted 5 checkpoint(s)",
  "count": 5
}
```

---

3. Recovery Scenarios

3.1 Scenario: Simple Recovery (Latest Checkpoint)

Situation: Agent crashed, need to recover from most recent checkpoint.

Steps:

1. Call recovery endpoint:

```
curl -X POST http://localhost:3002/api/recovery/recover \
  -H "Content-Type: application/json" \
  -d '{"agentId": "123e4567-e89b-12d3-a456-426614174000"}'
```

2. Use the restored state in your application to resume the agent.

3.2 Scenario: Recovery to Specific Point

Situation: Need to rollback to a known-good checkpoint.

Steps:

1. List available checkpoints to find the desired one by description or tags.
2. Recover to specific checkpoint by providing the checkpointId in the request.

3.3 Scenario: Recovery with Corrupted Checkpoint

Situation: Latest checkpoint is corrupted.

Steps:

1. Attempt recovery with fallback enabled (fallbackToLatest: true).
2. System will automatically detect corruption, mark the checkpoint as corrupted, and try older checkpoints until a valid one is found.

3.4 Scenario: Preventive Checkpointing

Situation: About to perform risky operation, want a save point.

Best Practice:

1. Create a checkpoint before the risky operation with descriptive tags.
2. Perform the risky operation in a try-catch block.
3. If operation fails, recover to the checkpoint you just created.

3.5 Scenario: Periodic Checkpointing

Situation: Long-running agent needs automatic checkpoints.

Implementation:

Use the PeriodicCheckpointManager class: 1. Create instance with checkpointManager and interval (e.g., 30000ms for 30 seconds) 2. Register agent with its ID and a function to get current state 3. Start the periodic manager 4. Stop when done

---

4. Troubleshooting

4.1 Connection Issues

Problem: MongoDB connection failed
Symptoms: Error: connect ECONNREFUSED 127.0.0.1:27017

Solutions:

1. Check MongoDB is running:
   - For Docker: docker ps | grep mongo
   - For local: mongosh –eval "db.adminCommand('ping')"
2. Verify connection string in .env file.
3. Check MongoDB logs:
   - For Docker: docker logs mongodb
   - For local: sudo tail -f /var/log/mongodb/mongod.log

Problem: Server not starting
Symptoms: Error: listen EADDRINUSE: address already in use :::3002

Solutions:

1. Find and kill existing process:
   - lsof -i :3002
   - kill –9 [PID]
2. Use different port:
   - PORT=3003 npm run dev

4.2 Checkpoint Issues

Problem: Checkpoint creation failed
Possible Causes:

1. State too large (default limit is 10MB)
2. Invalid state format (missing required fields, circular references)

Debugging:

Test serialization by calling JSON.stringify on your state. If it throws an error, you have a serialization issue.

Problem: Checkpoint skipped
Cause: No state changes detected since last checkpoint.

Solutions:

1. Check latest checkpoint to verify your expected changes
2. Make a small state change before creating checkpoint
3. Wait for actual state changes

4.3 Recovery Issues

Problem: No valid checkpoints found
Causes: 1. No checkpoints ever created 2. All checkpoints expired 3. All checkpoints corrupted

Solutions:

1. List all checkpoints to see what exists
2. Check for corrupted checkpoints using getRecoveryPoints
3. Verify TTL settings are not too aggressive

Problem: Base checkpoint not found
Cause: Incremental checkpoint references a deleted base.

Solutions:

1. Validate the checkpoint to find the issue
2. Delete orphaned incremental checkpoints
3. Prevent by never deleting base checkpoints

4.4 Performance Issues

Problem: Slow checkpoint creation
Solutions:

1. Use incremental checkpoints for frequent saves
2. Reduce state size by removing unnecessary data
3. Check MongoDB performance with stats command

Problem: Slow recovery
Causes: 1. Long incremental checkpoint chain 2. Large checkpoint data 3. MongoDB performance issues

Solutions:

1. Create periodic full checkpoints (every 10 incrementals)
2. Pre-validate checkpoints to skip invalid ones quickly

4.5 Common Error Messages

Error	Meaning	Solution
Invalid uuid	Agent/checkpoint ID format wrong	Use proper UUID format
Serialization failed	State cannot be JSON-serialized	Check for circular refs, functions
Integrity verification failed	Checkpoint data corrupted	Use fallback recovery
Recovery failed after all retries	Multiple attempts failed	Check checkpoint chain integrity
State size exceeds limit	State too large (>10MB)	Reduce state or increase limit

4.6 Getting Help

If you encounter issues not covered here:

1. Check logs by running: npm run dev 2>&1 | tee server.log
2. Enable debug mode: DEBUG=* npm run dev

3. Run tests: npm test
4. Check GitHub issues in the project repository

Document End

User Manual for Graduation Project - OS Concepts Applied to AI Agents